COMSATS UNIVERSITY ISLAMABAD

(ABBOTTABAD CAMPUS)

LAB – 1

MAIN TOPIC: **Faults, Failures and Errors**

COURSE: **SOFTWARE QUALITY ENGINEERING**

COURSE TEACHER: **SIR. MUKHTIAR ZAMIN**

SUBMITTED BY: **MUHAMMAD KHURRAM SHEHZAD**

REGISTRATION NO.: **FA23-BSE-070**

CLASS: **BSE-6B**

SUBMITTED ON: 7TH APRIL, 2026

TABLE OF CONTENTS

LAB TASK 1: **3**

SOLUTION: **3**

CODE ANALYSIS: 3
IDENTIFIED FAULTS (DEFECTS IN SOURCE CODE): 3
ERRORS RESULTING FROM FAULTS (COMPILE-TIME): 3
FAILURES (RUNTIME – AFTER FIXING COMPILATION ERRORS): 4
FAILURE TEST CASES (ACTUAL ≠ EXPECTED): 4
NON-FAILURE TEST CASES (ACTUAL = EXPECTED): 4
CORRECTED CODE: 5

LAB TASK 2: **5**

SOLUTION: **6**

INTRODUCTION: 6
Point Class Code: *6*
Line Class Code: *6*
Main Function: *7*
FAILURE TEST CASES: 7
EXPLANATIONS OF FAILURE: 8
NON-FAILURE TEST CASES: 8
EXPLANATION OF NON-FAILURE (PASSING) TEST CASES: 8
ANALYSIS AND DISCUSSION: 9
1. Incorrect Formula *9*
2. Missing Function Call *9*
3. Null Reference *9*
4. Correct Execution *9*

LAB TASK 3: **9**

SOLUTION: **9**

DEFECT # 1: 9
DEFECT # 2: 10
DEFECT # 3: 11

LAB TASK 1:

EVALUATE SOFTWARE BY OBSERVING IT'S EXECUTION. WRITE TEST CASES BY INTRODUCING SOME ERRORS, FAULTS AND FAILURES TO EXPLAIN YOUR ANSWER….

SOLUTION:

CODE ANALYSIS:

The provided CourseResult and CourseResultRun classes contain multiple **faults** (static defects in the code). When compiled, these faults produce **errors** (compile-time issues). If the code were to run (after fixing compilation errors), certain conditions could lead to **failures** (runtime behavior not matching expected output).

IDENTIFIED FAULTS (DEFECTS IN SOURCE CODE):

#	Fault Description	Location
Fault-1	Incorrect keyword – Public class instead of public class (Java is case-sensitive)	Line 1
Fault-2	Missing double quotes around string literals in display() – e.g., "Student Name is: — + studentname should be "Student Name is: " + studentname	display() method
Fault-3	Using incorrect dash character — instead of double quotes " in concatenated strings	display() method
Fault-4	Field names mismatch – declared as studentname, coursename, grade but assigned as studentName, courseName (camelCase vs lowercase)	main method
Fault-5	Missing closing braces for both classes – leads to compilation error	End of file

ERRORS RESULTING FROM FAULTS (COMPILE-TIME):

- Illegal start of type (due to Public)
- ';' expected (due to missing quotes and malformed string)
- Cannot find symbol for c1.studentName because field is named studentname
- Reached end of file while parsing (missing closing braces)

FAILURES (RUNTIME – AFTER FIXING COMPILATION ERRORS):

Even after correcting syntax, certain inputs or missing assignments cause failures:

Failure Scenario	Expected Output	Actual Output
Object created without setting grade – c1.grade remains null	Grade is: A	Grade is: null
Setting empty string for student name – c1.studentname = ""	Student Name is: Ali	Student Name is: (blank)
Swapping grade values (e.g., assign "A+" but expect "A")	Grade is: A	Grade is: A+

FAILURE TEST CASES (ACTUAL ≠ EXPECTED):

Test Case ID	Input / Condition	Expected Output	Actual Output	Fault Injected
TC-F1	c1.grade not initialized	Grade is: A	Grade is: null	Missing assignment
TC-F2	display() uses wrong variable – grade instead of coursename	Course Name is: OOP	Course Name is: A	Logic fault in method
TC-F3	Concatenation order swapped – grade + "Student Name"	Student Name is: Ali	AliStudent Name is:	String concatenation fault

NON-FAILURE TEST CASES (ACTUAL = EXPECTED):

Test Case ID	Input / Condition	Expected Output	Actual Output	Status
TC-P1	All fields correctly set – name="Bilal", course="DSA", grade="B+"	Student Name is: Bilal Course Name is: DSA Grade is: B+	Student Name is: Bilal Course Name is: DSA Grade is: B+	PASS
TC-P2	Grade value "A" with proper formatting	Grade is: A	Grade is: A	PASS
TC-P3	Course name with spaces – "Web Development"	Course Name is: Web Development	Course Name is: Web Development	PASS

CORRECTED CODE:

```
public class CourseResult {
    public String studentname;
    public String coursename;
    public String grade;

    public void display() {
        System.out.println("Student Name is: " + studentname +
                           " Course Name is: " + coursename +
                           " Grade is: " + grade);
    }
}

public class CourseResultRun {
    public static void main(String[] args) {
        CourseResult c1 = new CourseResult();
        c1.studentname = "Ali";
        c1.coursename = "OOP";
        c1.grade = "A";
        c1.display();

        CourseResult c2 = new CourseResult();
        c2.studentname = "Saba";
        c2.coursename = "ICP";
        c2.grade = "A+";
        c2.display();
    }
}
```

LAB TASK 2:

DESIGN A CLASS POINT WITH TWO DATA MEMBERS X-CORD AND Y-CORD. THIS CLASS SHOULD HAVE AN ARGUMENTS CONSTRUCTOR, SETTERS, GETTERS AND A DISPLAY FUNCTION. NOW CREATE ANOTHER CLASS —LINE||, WHICH CONTAINS TWO POINTS —STARTPOINT|| AND —ENDPOINT||. IT SHOULD HAVE A FUNCTION THAT FINDS THE LENGTH OF THE LINE. HINT: FORMULA IS: SQRT((X2-X1) 2 + (Y2- Y1) 2) CREATE TWO LINE OBJECTS IN RUNNER AND DISPLAY THE LENGTH OF EACH LINE. WRITE TEST CASES TO CHECK THE ABOVE FUNCTIONALITY BY INTRODUCING DIFFERENT ERRORS, FAULTS AND FAILURE MENTION 3 TEST CASES FOR FAILURES AND 3 TEST CASES FOR NON FAILURES AS SHOWN IN ABOVE EXAMPLE. ....

SOLUTION:

INTRODUCTION:

We design a **Point class** and a **Line class**. Each Point holds x and y coordinates with getters, setters, and a display method. The Line class has two points (startPoint and endPoint) and computes its length using the distance formula.

Point Class Code:

```
class Point {
    private double x;
    private double y;

    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }
    public double getX() { return x; }
    public double getY() { return y; }
    public void setX(double x) { this.x = x; }
    public void setY(double y) { this.y = y; }

    public void display() {
        System.out.println("Point: (" + x + ", " + y + ")");
    }
}
```

Line Class Code:

```
class Line {
    private Point startPoint;
    private Point endPoint;

    public Line(Point startPoint, Point endPoint) {
        this.startPoint = startPoint;
        this.endPoint   = endPoint;
    }

    // Calculate line length using Pythagorean theorem:
    public double getLength() {
        double x1 = startPoint.getX();
        double y1 = startPoint.getY();
        double x2 = endPoint.getX();
        double y2 = endPoint.getY();

        // distance = sqrt((x2 - x1)^2 + (y2 - y1)^2)
```

```
        return Math.sqrt(Math.pow(x2 - x1, 2) + Math.pow(y2 - y1, 2));
    }

    public void display() {
        System.out.println("Start Point:");
        startPoint.display();
        System.out.println("End Point:");
        endPoint.display();
        System.out.println("Length of Line: " + getLength());
    }
}
```

Main Function:

```
public class Main {
    public static void main(String[] args) {
        Point p1 = new Point(0, 0);
        Point p2 = new Point(3, 4);
        Line line1 = new Line(p1, p2);
        line1.display();

        Point p3 = new Point(2, 5);
        Point p4 = new Point(8, 5);
        Line line2 = new Line(p3, p4);
        line2.display();
    }
}
```

FAILURE TEST CASES:

The following test cases introduce faults to evaluate incorrect behavior:

Test Case	Expected Output	Actual Output	Result
1. Wrong operator in formula: Mistakenly use x1 + x2 instead of x2 - x1. For points (1,1) to (4,5).	Length = 5.0 (since 3-1=2, 5-1=4, √20 ≈ 4.47)	Length = 7.81 (if computed as √61 ≈ 7.81)	FAIL
2. Missing Math.sqrt: Return sum of squares only. For points (0,0) to (3,4).	Length = 5.0	Length = 25.0 (3^2+4^2)	FAIL
3. Null point reference: new Line(null, new Point(3,4))	Length = 5.0 (if handled) or error message	NullPointerException	FAIL

EXPLANATIONS OF FAILURE:

- **Case 1:**

  Using addition instead of subtraction is a logical fault.

  According to the formula, coordinates must be subtracted.

  The incorrect operation produces a wrong result (7.81 instead of correct value).

- **Case 2:**

  Missing Math.sqrt leads to returning the squared distance instead of actual distance.

  This significantly increases the output value.

- **Case 3:**

  Passing a null object causes a runtime failure.

  The program crashes when accessing methods on a null reference.

Each fault results in deviation between expected and actual output.

NON-FAILURE TEST CASES:

The following test cases confirm correct implementation:

Test Case	Expected Output	Actual Output	Result
1. Points (0,0) to (3,4)	Length = 5.0	Length = 5.0	PASS
2. Points (2,5) to (8,5)	Length = 6.0	Length = 6.0	PASS
3. Points (3,3) to (3,3)	Length = 0.0	Length = 0.0	PASS

EXPLANATION OF NON-FAILURE (PASSING) TEST CASES:

- The correct formula is applied.
- Valid, non-null inputs are used.
- Outputs match expected mathematical results exactly.
- Example: (0,0) → (3,4) forms a **3-4-5 triangle**, giving length = 5.

ANALYSIS AND DISCUSSION:

The Point and Line test cases demonstrate a fundamental testing principle:

1. Incorrect Formula

A small mistake (using + instead of -) significantly changes the result.

Correct implementation of the formula is essential.

2. Missing Function Call

Omitting Math.sqrt leads to incorrect output.

Even a small missing function can break correctness.

3. Null Reference

Passing null results in runtime failure.

This highlights the need for input validation.

4. Correct Execution

When inputs are valid and logic is correct, the system behaves as expected.

LAB TASK 3:

CONSIDER CUONLINE SYSTEM IDENTIFY, DOCUMENT, AND REPORT DEFECTS FOUND DURING TESTING....

SOLUTION:

Based on the testing of the CUOnline portal, following three defects have been identified. Each defect is documented below:

Defect # 1:

Module / Tab: Sidebar (Dashboard)

Defect Title: All sidebar tabs expand automatically on login, then collapse after few seconds

Description:

When a user logs into the CUOnline portal and lands on the Dashboard, all sidebar tabs/accordions are expanded at the same time. After a few seconds, they automatically close. The intended behavior is that all tabs should remain collapsed initially, and only expand when the user explicitly clicks on a specific tab.

Steps to Reproduce:

1. Navigate to CUOnline login page.
2. Enter valid credentials and log in.
3. Observe the sidebar immediately after dashboard loads.
4. Wait for 3–5 seconds.

Expected Result:

Sidebar tabs should collapse by default. Only the tab clicked by the user should expand, and it should remain expanded until another tab is clicked or the user collapses it.

Actual Result:

All tabs expand simultaneously upon page load, then automatically collapse after a short delay (approx. 3–5 seconds).

Environment: Browser: Chrome, Edge.

Defect # 2:

Module / Tab: Library Tab

Defect Title: Library tab does not display reserved books or borrow history when data exists

Description:

When a user clicks the "Library" tab in the sidebar, the page loads but shows "No reserved books" and "No borrow history", even though the user has active book reservations and a history of issued books. The tab should fetch and display the correct user-specific library data.

Steps to Reproduce:

1. Log into CUOnline as a student who has reserved at least one book and has borrow history.
2. Click on the "Library" tab in the sidebar.
3. Observe the sections "Reserved Books" and "Borrow History".

Expected Result:

The "Reserved Books" section should list all currently reserved books with details (title, author, reservation date). The "Borrow History" section should show past issued books with return dates.

Actual Result:

Both sections display empty state messages: "No reserved books" and "No borrow history", despite the user having valid reservations and history in the backend database.

Environment: Browser: Chrome, Edge.

Defect # 3:

Module / Tab: Graduate Progress Profile Tab

Defect Title: Clicking "Graduate Progress Profile" redirects to Dashboard instead of Progress Profile page

Description:

The sidebar contains a tab labeled "Graduate Progress Profile". When a student clicks on this tab, the intended navigation should take the student to their personal graduate progress profile (showing completed courses, GPA, degree completion percentage, etc.). However, the current behavior redirects the user back to the Dashboard instead.

Steps to Reproduce:

1. Log into CUOnline as a graduate student.
2. In the sidebar, locate the "Graduate Progress Profile" tab.
3. Click on the tab.
4. Observe the page that loads.

Expected Result:

The user should be navigated to the Graduate Progress Profile page, displaying relevant academic progress information.

Actual Result:

The user is redirected to the Dashboard (same page as after login). No progress profile information is shown.

Environment: Browser: Chrome, Edge.